설비 고장을 예측하는 AI 데이터 분석

리스트와 조건문

리스트

posco

설비 고장을 예측하는 AI 데이터 분석

리스트와 조건문

리스트 Part 1

리스트 만들기

인덱싱과 슬라이싱

리스트 값 추가·제거

조건문 Part 2

if 문의 기본 구조

if-elif-else 전체 구조

리스트

posco

여러 값을 따로 관리할 때의 불편함

변수 하나에 값 하나 - 측정값 100개면 변수 100개 필요

평균·최댓값 계산 시 코드가 값 개수만큼 길어짐

여러 값을 하나로 묶어 다루는 도구가 필요

리스트 - 값을 담는 목록

여러 값 담기 - 변수와 달리 여러 값을 한 목록에 보관

순서 유지 - 담은 순서가 그대로 유지돼 나중에 찾기 쉬움

리스트 만들기와 출력

대괄호 [] 안에 값을 쉼표로 나열해 생성

CODE

```
# 숫자 리스트
temps = [25, 26, 24, 28, 27]
print(temps)        # [25, 26, 24, 28, 27]

# 글자 리스트 (따옴표 필수)
machines = ["펌프", "모터", "압축기"]
print(machines)     # ['펌프', '모터', '압축기']
```

리스트에 담는 여러 종류의 값

리스트는 담는 값의 종류를 가리지 않음

1. 숫자와 글자, 참거짓 값을 한 리스트에 섞어 담기 가능

2. 다만 분석 실무에서는 **같은 종류의 값만 담는** 경우가 대부분

빈 리스트와 길이 len()

빈 리스트로 시작할 수 있고, **len()** 으로 담긴 값의 개수 확인

CODE

```
temps = [25, 26, 24, 28, 27]
print(len(temps))   # 5  (담긴 값의 개수)

results = []         # 빈 리스트
print(len(results))  # 0
```

리스트와 변수의 차이

구분	변수	리스트
담는 값	값 하나	여러 값
새 값 넣으면	이전 값 사라짐	옆에 추가됨
순서	없음	담은 순서 유지
비유	받침대	서랍장

실습 1. 나만의 데이터 리스트 만들기

목표	여러 값을 대괄호와 쉼표로 리스트에 담고, 출력과 len() 으로 개수 확인하기
단계	① 하루 측정 온도 5개를 대괄호 안에 쉼표로 구분해 리스트로 저장 → ② print 로 전체 출력 → ③ len 으로 값 개수를 출력해 5개인지 확인 → ④ 빈 리스트를 만들어 len 이 0인지 확인
예상 결과	[25, 26, 24, 27, 26] / 5 / [] / 0

실습 1. 나만의 데이터 리스트 만들기 정답

CODE

```
temps = [25, 26, 24, 27, 26]
print(temps)          # [25, 26, 24, 27, 26]
print(len(temps))     # 5
empty = []
print(len(empty))     # 0
```

인덱스 - 순서로 값 찾기

리스트 이름 뒤 대괄호에 순서 번호(인덱스)를 넣어 값 하나를 꺼냄

CODE

```
temps = [25, 26, 24, 28, 27]
print(temps[0])   # 25  (첫 번째 값)
print(temps[2])   # 24  (세 번째 값)
```

인덱스는 0부터 시작

첫 번째 값이 **인덱스 0** - 마지막 인덱스는 (값 개수 - 1)



음수 인덱스로 뒤에서 찾기

인덱스에 음수를 쓰면 뒤에서부터 - 마지막은 **-1**

CODE

```
temps = [25, 26, 24, 28, 27]
#        -5  -4  -3  -2  -1
print(temps[-1])   # 27  (마지막 값)
print(temps[-2])   # 28  (뒤에서 두 번째)
```

인덱스 범위를 벗어나면

IndexError - 없는 자리를 가리키면 'index out of range' 오류 발생

점검 - 마지막 인덱스는 (개수-1), 무심코 하나 크게 넣지 않았는지 확인

실습 2. 인덱스로 값 꺼내기

목표	양수·음수 인덱스로 원하는 위치의 값을 정확히 꺼내기
단계	① 시간대 온도 6개를 리스트로 저장 → ② 첫 번째 값을 인덱스 0으로 꺼내 출력 → ③ 세 번째 값을 꺼내 출력 → ④ 마지막 값을 음수 인덱스로 꺼내 출력
예상 결과	22 / 27 / 23

실습 2. 인덱스로 값 꺼내기 정답

CODE

```
temps = [22, 24, 27, 29, 26, 23]
print(temps[0])    # 22
print(temps[2])    # 27
print(temps[-1])   # 23
```

실습 3. 인덱스로 꺼낸 값 계산하기

목표	인덱스로 꺼낸 값을 변수에 담아 더하고 중간값 구하기
단계	① 라인 생산량 6개를 리스트로 저장 → ② 첫 값과 마지막 값을 각각 인덱스로 꺼내 변수에 담기 → ③ 두 값을 더한 합을 출력 → ④ 두 값을 더해 2로 나눈 중간값을 출력
예상 결과	222 / 111.0

실습 3. 인덱스로 꺼낸 값 계산하기 정답

CODE

```
output = [120, 95, 130, 110, 88, 102]
first = output[0]
last = output[-1]
print(first + last)          # 222
print((first + last) / 2)    # 111.0
```

슬라이싱 - 구간 잘라내기

콜론으로 범위 지정 - **끝 인덱스는 미포함**, 결과는 새 리스트

CODE

```
temps = [25, 26, 24, 28, 27]
#         0   1   2   3   4
print(temps[1:4])   # [26, 24, 28]
# 인덱스 1,2,3만 (4는 미포함) → 4-1=3개
```

슬라이싱 시작·끝 생략

시작 생략은 처음부터, 끝 생략은 끝까지, 둘 다 생략은 전체

CODE

```
temps = [25, 26, 24, 28, 27]
print(temps[:3])   # [25, 26, 24]  처음부터
print(temps[2:])   # [24, 28, 27]  끝까지
print(temps[:])    # 전체
print(temps[-3:])  # [24, 28, 27]  뒤에서 3개
```

슬라이싱 간격 step

콜론을 하나 더 붙여 간격(step) 지정 - 값을 건너뛰며 가져오기

CODE

```
temps = [25, 26, 24, 28, 27, 23]
print(temps[0:6:2])   # [25, 24, 27]  2칸씩
print(temps[::2])     # [25, 24, 27]  전체에서 2칸씩
```

인덱싱·슬라이싱 구조 정리

콜론이 있으면 슬라이싱(리스트), 없으면 인덱싱(값 하나)

구분	인덱싱	슬라이싱
표기	temps[2]	temps[1:4]
콜론	없음	있음
결과	값 하나 24	리스트 [26,24,28]
끝 포함	해당없음	끝은 미포함

실습 4. 슬라이싱으로 구간 자르기

목표	시작·끝·생략 규칙으로 원하는 구간을 잘라내고 개수 확인하기
단계	① 측정 온도 10개를 리스트로 저장 → ② 앞에서 3개를 슬라이싱해 출력 → ③ 뒤에서 3개를 슬라이싱해 출력 → ④ 잘라낸 앞 구간에 len 을 적용해 3개인지 확인
예상 결과	[22, 24, 27] / [28, 30, 21] / 3

실습 4. 슬라이싱으로 구간 자르기 정답

CODE

```
temps = [22, 24, 27, 29, 26, 23, 25, 28, 30, 21]
print(temps[:3])        # [22, 24, 27]
print(temps[-3:])       # [28, 30, 21]
print(len(temps[:3]))   # 3
```

실습 5. 데이터를 두 구간으로 나누기

목표	데이터를 앞·뒤 두 구간으로 나누고 각 구간을 따로 다루기
단계	① 12개 값을 리스트로 저장 → ② 앞 6개를 슬라이싱해 first 에 담고 출력 → ③ 뒤 6개를 슬라이싱해 second 에 담고 출력 → ④ 두 구간에 각각 len 을 적용해 6개씩인지 확인
예상 결과	[3, 4, 5, 6, 5, 4] / [6, 7, 8, 7, 5, 3] / 6 6

실습 5. 데이터를 두 구간으로 나누기 정답

CODE

```
hours = [3, 4, 5, 6, 5, 4, 6, 7, 8, 7, 5, 3]
first = hours[:6]
second = hours[6:]
print(first)          # [3, 4, 5, 6, 5, 4]
print(second)         # [6, 7, 8, 7, 5, 3]
print(len(first), len(second))   # 6 6
```

인덱스로 값 바꾸기

등호 왼쪽에 인덱스를 두면 그 자리의 값만 교체

CODE

```
temps = [25, 26, 24, 28, 27]
temps[2] = 99       # 인덱스 2의 값 교체
print(temps)        # [25, 26, 99, 28, 27]
```

값이 들어있는지 확인 in

in 연산자로 값의 존재를 참·거짓으로 확인

CODE

```
machines = ["펌프", "모터", "압축기"]
print("모터" in machines)     # True
print("밸브" in machines)     # False
print("밸브" not in machines) # True
```

실습 6. 값 찾아 바꾸기

목표	in 으로 값의 존재를 확인하고 인덱스로 값 바로잡기
단계	① 온도 리스트를 저장(하나는 잘못된 값 240) → ② in 으로 잘못된 값이 있는지 확인 → ③ 그 값의 위치를 index 로 찾아 올바른 값 24를 대입 → ④ 전체를 출력하고 in 으로 다시 확인
예상 결과	True / [25, 26, 24, 28, 27] / False

실습 6. 값 찾아 바꾸기 정답

CODE

```
temps = [25, 26, 240, 28, 27]
print(240 in temps)      # True
i = temps.index(240)
temps[i] = 24
print(temps)             # [25, 26, 24, 28, 27]
print(240 in temps)      # False
```

append - 끝에 값 추가

append() 는 리스트 맨 끝에 값 하나를 추가

CODE

```
temps = [25, 26, 24]
temps.append(28)
print(temps)        # [25, 26, 24, 28]
temps.append(27)
print(temps)        # [25, 26, 24, 28, 27]
```

insert - 원하는 위치에 삽입

insert(위치, 값) 은 지정 위치에 끼워 넣고 뒤 값은 밀림

CODE

```
temps = [25, 26, 24, 27]
temps.insert(2, 99)     # 인덱스 2에 99 삽입
print(temps)            # [25, 26, 99, 24, 27]
```

extend - 리스트 이어붙이기

extend() 는 다른 리스트의 값들을 풀어서 이어붙임

CODE

```
morning = [25, 26, 24]
afternoon = [28, 27, 30]
morning.extend(afternoon)
print(morning)   # [25, 26, 24, 28, 27, 30]
```

실습 7. 측정값 추가하기

목표	append·insert·extend 를 상황에 맞게 사용해 값 추가하기
단계	① 빈 리스트로 시작 → ② append 로 값 하나를 끝에 추가 → ③ insert 로 맨 앞에 값 끼워 넣기 → ④ 준비된 묶음을 extend 로 이어붙여 전체 출력
예상 결과	[30] / [28, 30] / [28, 30, 31, 32]

실습 7. 측정값 추가하기 정답

CODE

```
temps = []
temps.append(30)
print(temps)          # [30]
temps.insert(0, 28)
print(temps)          # [28, 30]
temps.extend([31, 32])
print(temps)          # [28, 30, 31, 32]
```

remove - 값으로 제거

remove(값) 은 그 값과 같은 첫 번째 항목을 제거

CODE

```
machines = ["펌프", "모터", "압축기", "모터"]
machines.remove("모터")
print(machines)   # ['펌프', '압축기', '모터']
```

pop - 위치로 꺼내며 제거

pop(위치) 는 그 자리 값을 꺼내 돌려주면서 제거

CODE

```
temps = [25, 26, 24, 28, 27]
removed = temps.pop(2)   # 인덱스 2 제거
print(removed)           # 24  (꺼낸 값)
print(temps)             # [25, 26, 28, 27]
```

del - 인덱스로 삭제

del 은 인덱스로 삭제하되 값을 돌려주지 않음 (점 없이 사용)

CODE

```
temps = [25, 26, 24, 28, 27]
del temps[2]        # 인덱스 2 삭제
print(temps)        # [25, 26, 28, 27]
# del temps[1:3]    # 구간 삭제도 가능
```

실습 8. 잘못된 값 제거하기

목표	remove·pop·del 을 기준(값·위치)에 맞게 사용해 값 제거하기
단계	① 이상값 999가 있는 리스트 저장 → ② remove 로 이상값을 값 기준으로 제거 → ③ pop 으로 특정 위치 값을 꺼내며 제거해 꺼낸 값 출력 → ④ del 로 첫 번째 값을 삭제하고 전체 출력
예상 결과	[25, 26, 24, 28, 26] / 꺼낸 값 26 / [24, 28, 26]

실습 8. 잘못된 값 제거하기 정답

CODE

```
temps = [25, 26, 999, 24, 28, 26]
temps.remove(999)
print(temps)          # [25, 26, 24, 28, 26]
x = temps.pop(1)
print(x)              # 26
del temps[0]
print(temps)          # [24, 28, 26]
```

sort - 정렬하기

sort() 는 작은 값부터(오름차순), reverse=True면 큰 값부터

CODE

```
temps = [27, 24, 30, 22, 26]
temps.sort()
print(temps)              # [22, 24, 26, 27, 30]
temps.sort(reverse=True)
print(temps)              # [30, 27, 26, 24, 22]
```

reverse - 순서 뒤집기

reverse() 는 크기와 무관하게 현재 순서를 그대로 뒤집음

CODE

```
temps = [25, 30, 22, 28]
temps.reverse()
print(temps)        # [28, 22, 30, 25]
```

count·index - 세고 찾기

count(값) 은 몇 번 나오는지, **index(값)** 은 처음 나오는 위치

CODE

```
results = ["정상", "이상", "정상", "정상", "이상"]
print(results.count("정상"))   # 3  (몇 개)
print(results.index("이상"))   # 1  (몇 번째)
```

리스트 메서드 한눈에 정리

용도	메서드	하는 일
추가	append/insert/extend	끝/위치/묶음 잇기
제거	remove/pop/del	값/위치+ 반환/위치
정렬	sort/reverse	크기순/뒤집기
탐색	count/index	개수/처음 위치

실습 9. 정렬하고 탐색하기

목표	sort·reverse·count·index 로 리스트를 정렬하고 값을 탐색하기
단계	① 온도 리스트를 저장 → ② sort 로 오름차순 정렬해 출력 → ③ reverse 로 뒤집어 출력 → ④ count 로 특정 값의 개수와 index 로 그 값의 처음 위치를 출력
예상 결과	[22, 24, 24, 26, 27, 28, 30] / [30, 28, 27, 26, 24, 24, 22] / 2 / 4

실습 9. 정렬하고 탐색하기 정답

CODE

```
temps = [27, 24, 30, 22, 26, 24, 28]
temps.sort()
print(temps)            # [22, 24, 24, 26, 27, 28, 30]
temps.reverse()
print(temps)            # [30, 28, 27, 26, 24, 24, 22]
print(temps.count(24))  # 2
print(temps.index(24))  # 4
```

감사합니다.

posco